

Reproducible Project Workflows

Git, GitHub, and getting started with the lab project template

Irina Gaynanova

Table of contents

1	What reproducibility means	1
2	Version control with Git	2
2.1	Configuration	2
2.2	A few terminal commands	2
2.3	One project, one repository	2
2.4	The workflow: raw, staged, committed	2
2.5	Commit practices	3
3	GitHub	4
3.1	Two directions	4
3.2	Staying in sync	4
3.3	Collaborative workflow	5
3.4	Conflict resolution	5
3.5	Useful extras	5
4	R projects	5
5	Project structure: the lab template	6
5.1	Three variants	6
5.2	The core principles	6
5.3	Two that bite first	7
6	Recording the computational environment	8
	Checklist	8
	Further reading	8
	License	9

1 What reproducibility means

A concrete, testable definition:

If someone else clones your repository onto their machine, opens an .R file and runs it, the code should **produce no errors** and **produce output that matches yours exactly**.

Both halves matter. Code that errors on another machine is not reproducible, but neither is code that runs and silently produces different numbers. This standard is strict enough to be useful: it can be checked, and almost every common reproducibility failure shows up when you try.

The practical test is easy to run and worth running before any deadline: clone your own repository into a fresh directory, open it, and run everything from the top. Anything that breaks is something a collaborator, a reviewer, or you in six months would have hit.

Getting there takes two things. The first is **version control** — Git and GitHub — which is what §2 and §3 cover, from installation onward. The second is **project structure**: where files live and what order things run in. For that, the lab maintains a template repository, and §5 introduces it rather than restating it here.

2 Version control with Git

Git is a version control system: it tracks changes to code and documents over time, letting you see what changed, when, and why, and return to any earlier state. Without it, version history lives in filenames — `analysis_v2_final.R`, `analysis_v2_final_REAL.R` — which records nothing about what actually changed and collapses entirely once more than one person is involved.

Check that Git is installed:

```
git --version
```

2.1 Configuration

Git labels every change with an author, so tell it who you are once:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

2.2 A few terminal commands

Git is usually driven from RStudio, but a handful of shell commands are needed to get started:

Command	What it does
<code>ls -a</code>	List files in the current directory, including hidden ones
<code>cd folder</code>	Move into folder
<code>cd</code>	Return to the home directory
<code>cd ..</code>	Move one directory up

Hidden files matter here: `.git`, `.gitignore`, and `.Rproj.user` all begin with a dot and are invisible to a plain `ls`.

2.3 One project, one repository

The organizing principle:

One project → one repository → one `.Rproj` → one `.gitignore`.

Create a repository by moving to the desired location and initializing it:

```
git init ProjectName
cd ProjectName
git status
```

Never nest a version-controlled repository inside another one. Git will track the outer repository and treat the inner one as an opaque entry, and the history of the inner project effectively disappears.

2.4 The workflow: raw, staged, committed

Changes move through three states:

raw changes → **staged changes** (`git add`) → **committed changes** (`git commit`)

A **commit** is a snapshot of the project that includes all staged changes and only those. Every difference Git reports is measured against the most recent commit.

In the RStudio Git pane, a changed file appears as untracked (?? for new files, M for modified ones) until you stage it, at which point its status becomes A. The same operations from the terminal:

```
git status          # what has changed, staged and unstaged
git add readme.md   # stage a file
git commit -m "Start of readme" # commit everything staged
git log             # view history
```

Staging and committing often happen together, but they are separate steps for a reason: staging lets you commit *some* of your current changes and leave the rest for a later, separate commit.

Files must be **saved** before Git sees the change. An unsaved editor buffer shows nothing in the Git pane.

2.5 Commit practices

Two habits determine whether a project's history is useful or worthless.

2.5.1 Write informative commit messages

A message should complete this sentence:

If applied, this commit will ____

Good messages, with only the bold part being the actual commit text:

- If applied, this commit will **update getting started documentation**
- If applied, this commit will **fix bug associated with data input**
- If applied, this commit will **add function generateY**

Messages that fail the test, because they describe activity rather than change: “fixed a bug”, “changed a few things”, “more code adjustments”, “update”. At minimum, a reader should be able to guess what happened in a commit from its message alone.

2.5.2 Commit frequently and logically

- **Avoid large bulk commits.** Break work into smaller pieces. A commit containing an entire assignment is not a history.
- **One logical change per commit.** Two unrelated bug fixes are two commits. Fixing something and reformatting something else at the same time makes both harder to review and impossible to revert independently.
- **Do not commit half-done or untested work.** “Started to fix bug X” is not a snapshot worth keeping.

It is fine to commit several functions together if they are small, or a function alongside its tests, because those belong together logically.

A history that says nothing:

```
Started assignment  
Made changes  
Finished assignment
```

A history that says everything:

```
add student's name  
add screenshots  
finish first version of function X  
correct bug on variance in function X
```

3 GitHub

Git alone does version control locally, which is enough if you are the only person on the project and your laptop never fails. Collaboration and backup require a remote host. GitHub, GitLab, and Bitbucket all serve this purpose; these materials use GitHub.

GitHub mirrors your local repositories, accepts changes you upload, and lets you pull down changes made elsewhere — serving as both a collaboration hub and a cloud backup.

3.1 Two directions

From GitHub to local — a repository already exists on GitHub and you want a copy to work on. **Fork** it first if it belongs to someone else (this creates your own copy under your account), then clone:

```
git clone [repository URL]
```

In RStudio: *New Project* → *Version Control* → *Git*, then enter the URL. You can now open the local .Rproj and start working.

From local to GitHub — you have a local repository and want to mirror it. Create a new repository on GitHub, **without initializing it with a README** (that would create a conflicting history), then link and push:

```
git remote add origin [URL of the GitHub repository]  
git push --set-upstream origin main
```

It is strongly recommended that the GitHub repository name match your local folder name.

Either way, you end up with a local repository and its mirror on GitHub.

3.2 Staying in sync

Once local and remote match, they drift apart as soon as either side changes.

Local changes → GitHub:

```
git push origin main
```

GitHub changes → local (someone else's work, or your own edits made through the GitHub web interface):

```
git pull origin main
```

3.3 Collaborative workflow

With several people committing to one repository at once, the routine is:

1. **Pull first** to get everyone else's updates.
2. Make your changes and commit them **frequently**.
3. **Push often** so others are working against current code.
4. Resolve any conflicts.

Pushing rarely is what causes painful conflicts; the longer two branches of work diverge, the more there is to reconcile.

3.4 Conflict resolution

A conflict occurs when the same file has been changed both locally and remotely since the last sync. Git will refuse the push. The resolution order is:

pull → merge (decide what to keep) → commit → push

Git marks conflicting regions inside the file; edit it so it contains the version you want, commit that decision, and push. Conflicts are routine, not a sign that something has gone wrong.

3.5 Useful extras

Public and private repositories. Repositories are public by default; work in progress usually belongs in a private one.

Forks. A fork is a copy of someone else's public repository under your account, at a point in time, with full administrative rights. Useful as a starter for something new, and as the basis for proposing changes back to the original repository via a **pull request**.

Issues. How users report problems with your software — a lightweight tracker attached to the repository.

Branches. A way to develop edits separately from the main line of development. Powerful for larger projects; for a small project a single branch is usually simpler.

4 R projects

Use an RStudio project (`.Rproj`) as the unit of work. Projects:

- support the one-project-one-repository principle;
- set the working directory to the project root automatically, which is what makes relative paths work;
- integrate directly with Git through the RStudio Git pane.

To create a project in a directory that already has a repository: *File → New Project → Existing Directory*, and choose it. A Git tab should appear alongside Environment and History. If it does not, restart RStudio; if it still does not, enable version control under *Tools → Global Options → Git/SVN*.

Two new files appear immediately: the `.Rproj` file itself, and `.gitignore`, which lists files Git should not track. RStudio populates `.gitignore` with the hidden files associated with the project. Add to it anything that is generated rather than authored — large intermediate data, rendered output you do not want versioned, OS clutter such as `.DS_Store`.

5 Project structure: the lab template

Everything above is about *version control* — the machinery for tracking and sharing work. Structure is the other half: where files go, what order scripts run in, and which conventions make an analysis legible to someone else. For that, the lab maintains a template repository:

IrinaStatsLab/IrinaStatsLab-project-template

Click *Use this template* on GitHub to create your own copy, clone it, and open the .Rproj file. Its README is the authoritative statement of lab practice — this section is an orientation to it, not a replacement.

The template goes well beyond a simulation study. It is built for applied data analysis and for longer, more complex projects: work with raw and processed data, multi-stage pipelines, parallel analysis threads, and written reports.

5.1 Three variants

The template is not one fixed layout. R/ (reusable functions) and output/ (figures, tables, intermediate objects) are present in every project; everything else depends on the kind of work:

Variant	For	Adds
Data analysis	Projects centered on an external dataset	data/raw/ and data/processed/, scripts/ for setup and utilities, analysis/ for the numbered pipeline, reports/
Methods development	Projects where simulations are the engine, real data secondary	simulations/ with configs/, numbered run and aggregate scripts, and results/
Lightweight	Smaller or exploratory projects	data/ and R/ only; no formal pipeline

A simulation study of the kind covered in the companion simulation guide is a **methods development** project. The flat structure used in the small worked example is the same separation of concerns, scaled down.

5.2 The core principles

The template distinguishes **core** practices — do these from day one, because violations cause immediate, concrete problems — from **build-toward** practices that matter more as a project grows. The ten core ones:

Principle	In short
File paths are relative to the project root	Code runs unchanged on any machine
Code runs without manual intervention	No editing paths, no comment-in/comment-out switches, no <code>setwd()</code>
Raw data is never modified	data/raw/ is read-only; transformations write elsewhere
Figures and tables are generated by code	Never export from the Plots pane; PDF by default
Results are exactly reproducible	Explicit seeds, no hidden session state, fresh-session test
Scripts are modular and run in sequence	Numbered stages, each independently runnable

Principle	In short
Intermediate outputs are saved	Each stage writes to disk; the next loads rather than recomputes
Each analytical step is done exactly once	Logic lives in one place; downstream consumes its output
Analytical decisions are documented	Comments explain <i>why</i> , at the point of the decision
The project is self-contained	Anyone cloning it can find the data or instructions to get it

The build-toward practices cover version control itself, data validation at each pipeline stage, naming conventions, separating functions from scripts, separating computation from presentation, and code review before submission. The template's README gives the reasoning and R implementation details for each, along with notes on when each one starts to pay off.

5.3 Two that bite first

Two core principles cause more trouble for a first project than the rest combined, and both connect directly to the R project setup above.

Relative paths. Suppose `02_model.R` lives in `analysis/` and needs a function defined in `R/model_functions.R`. Three ways to get it wrong:

```
# (a) Calling the function without sourcing anything.
out <- fit_model(X)
# Works only if the function happens to be in your environment already.

# (b) Sourcing by absolute path.
source("/Users/Irina/Documents/MyProject/R/model_functions.R")
# That path exists on exactly one computer.

# (c) Sourcing relative to the script's own folder.
source("model_functions.R")
# Paths resolve from the project root, not from the script's location.
```

The correct version, from anywhere in the project:

```
source("R/model_functions.R")
out <- fit_model(X)
```

This is what opening the `.Rproj` file buys you: the working directory is always the project root, so the same relative path resolves on every machine. Never use `setwd()`.

Outputs generated by code. Saving a figure through the RStudio *Export* button, or typing numbers into a table by hand, breaks reproducibility at the last step — nobody can regenerate what you produced, including you.

```
pdf("output/figures/comparison.pdf", width = 6, height = 4)
print(p)
dev.off()
```

PDF is the default because it is vector-based and accepted everywhere; PNG is not publication quality. Build tables programmatically too, with `knitr::kable()` or similar, rather than writing them out by hand.

6 Recording the computational environment

Code depends on the versions of the packages it uses, and package behavior changes between versions. A result you cannot attribute to specific versions is a result you cannot defend later.

For a short project, recording `sessionInfo()` output alongside your results is a reasonable start. For anything that will outlive a few weeks or involve collaborators, use `renv`:

```
renv::init()      # start tracking this project's packages
renv::snapshot()  # record current versions into renv.lock
renv::restore()   # reproduce that library on another machine
```

Commit `renv.lock`; collaborators run `renv::restore()` to recreate your library.

One RStudio setting is worth changing immediately: under *Tools* → *Global Options* → *General*, disable “Restore .RData into workspace at startup” and set “Save workspace to .RData on exit” to **Never**. The environment should always be rebuilt from code, never carried over from a previous session — otherwise a script can appear to work only because of an object left over from an hour ago.

Checklist

Version control:

- ☐ Git installed and configured with your name and email
- ☐ One project, one repository, one `.Rproj`, one `.gitignore`
- ☐ No repository nested inside another
- ☐ Commit messages complete “If applied, this commit will ____”
- ☐ Commits are frequent, each covering one logical change
- ☐ No half-done or untested work committed
- ☐ Repository mirrored on GitHub; pushed regularly
- ☐ Pull before starting work; push when finishing

Project structure — see the template for the full list:

- ☐ Project started from the lab template, correct variant chosen
- ☐ All paths relative to the project root; no `setwd()`
- ☐ Raw data never modified
- ☐ Scripts numbered, modular, each independently runnable
- ☐ Intermediate outputs saved to disk
- ☐ Every figure and table produced by a script
- ☐ Seeds set explicitly; `sessionInfo()` or `renv.lock` recorded
- ☐ Fresh clone, fresh session: runs top to bottom, output matches

Further reading

[IrinaStatsLab project template](#) — the lab’s structure and principles for R projects, with reasoning and implementation details for each practice.

[Happy Git and GitHub for the useR](#) — Jenny Bryan’s book; the best resource for installation, authentication, and troubleshooting when Git and RStudio will not cooperate.

[Software Carpentry: Version Control with Git](#) — a hands-on introduction covering the core workflow from the ground up.

[How to Write a Git Commit Message](#) — Chris Beams; the source of the “If applied, this commit will” rule and a short argument for why commit messages deserve care.

[GitHub Skills](#) — free interactive courses run inside real repositories.

[Tidyverse style guide](#) — the naming and formatting conventions the lab template follows.

Wilson G, Bryan J, Cranston K, Kitze J, Nederbragt L, Teal TK (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510> — project organization, file naming, and data management, aimed at researchers rather than software engineers.

License

“Reproducible Project Workflows” © 2026 by Irina Gaynanova is licensed under the [Creative Commons Attribution 4.0 International License \(CC BY 4.0\)](#). You are free to share and adapt this guide for any purpose, including commercially, provided you give appropriate credit, link to the license, and indicate if changes were made. Suggested attribution: “Adapted from Reproducible Project Workflows by Irina Gaynanova, CC BY 4.0.”